

LexComply Intelligence

Legal & Compliance Research Assistant – Proof of Concept Documentation

Document type	Technical & Functional POC Reference
Version	1.0
Date	6 October 2026
Repository	legal-compliance-assistant
Audience	Developers, architects, interviewers, future maintainers

Contents

1. Executive Summary
2. Objectives and Scope
3. Technology Stack
4. System Architecture
5. Backend Microservices
6. Data Model
7. Frontend Architecture
8. Feature Catalogue
9. Intelligence Engines (Algorithms & Formulas)
10. Role-Based Views and Access Matrix
11. Configuration Reference
12. Running the POC Locally
13. Testing and Quality
14. Known Limitations and Honest Disclosures
15. Demo Script
16. Interview Talking Points and Q&A
17. Future Roadmap (Next-Generation GRC + AI Platform)
18. File Inventory
19. Glossary

1. Executive Summary

LexComply Intelligence is a Legal & Compliance Research Assistant built as a microservices proof of concept. It manages policies, regulations, compliance assessments, reports and users, and layers a **compliance intelligence platform** on top: an Executive Command Center, an explainable Risk Scoring Engine, a Compliance Health Score, a Risk Heatmap, Gap Analysis, a Compliance Maturity Index, a Policy Approval Workflow, an Audit Timeline, an Executive PDF Report, Role-Based Dashboards, Global Search (⌘K), Dark Mode and an AI Compliance Copilot.

The platform is positioned in the same product category as Microsoft Purview Compliance Manager, ServiceNow GRC, RSA Archer and MetricStream, and the documentation below explains how each capability works, where it lives in the code, and how it can evolve into a full AI-driven GRC product.

Key outcomes

- Five Spring Boot microservices behind a Spring Cloud Gateway, backed by PostgreSQL, documented with Swagger/OpenAPI.
- A React enterprise shell (sidebar, top bar, command palette, Copilot drawer) with live data refreshed every 60 seconds.
- A pure, unit-tested intelligence layer (risk, maturity, gaps, recommendations, workflow, copilot) with zero new npm dependencies.
- Production build compiles with no ESLint warnings; 10 automated tests pass.

2. Objectives and Scope

2.1 Objectives

1. Demonstrate an end-to-end microservices architecture (React → API Gateway → services → PostgreSQL).
2. Transform a CRUD application into a decision-support product for executives, compliance officers, auditors and administrators.
3. Make every score explainable (no “black box” numbers).
4. Be AI-ready: define a contract for a future Retrieval-Augmented Generation (RAG) service.

2.2 In scope

- Registers: Policies, Regulations, Compliance Assessments, Reports, Users.
- Intelligence: health, risk, maturity, gaps, recommendations, heatmap, analytics, timeline, workflow, executive report, copilot.
- UX: enterprise layout, responsive design, dark mode, global search, role-based dashboards, print-ready reports.

2.3 Out of scope (POC)

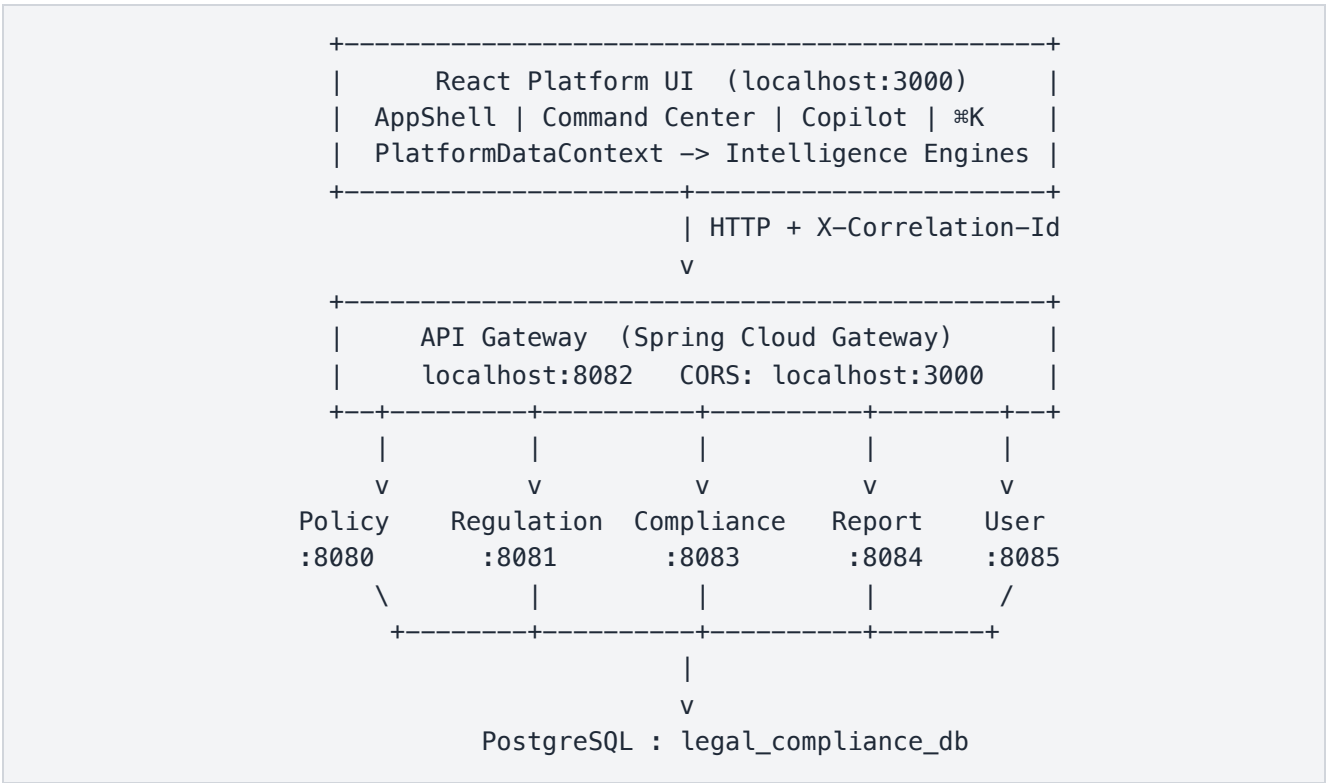
- Authentication/authorization (JWT, Keycloak) – role switcher is a UI persona only.
- Server-side persistence of workflow decisions and risk scores.
- LLM / vector search integration (contract defined, not deployed).

3. Technology Stack

Layer	Technology	Version / Notes
Frontend framework	React	19.x (Create React App, react-scripts 5.0.1)
Routing	React Router DOM	7.x (nested routes with layout <Outlet/>)
UI toolkit	Bootstrap	5.3.x (native data-bs-theme dark mode)
HTTP	Axios	1.x, shared client with interceptors
Charts	Custom SVG components	Gauge, Donut, BarList, Radar – no chart library
Backend	Spring Boot	4.1.1, Java 21
Gateway	Spring Cloud Gateway (WebMVC)	Spring Cloud 2025.1.3
Persistence	Spring Data JPA + PostgreSQL	DB legal_compliance_db, ddl-auto=update
Validation	Jakarta Bean Validation	@NotBlank, @Email
API docs	springdoc-openapi	Swagger UI per service
Testing	Jest + React Testing Library	Engine unit tests + App smoke test

4. System Architecture

4.1 Logical architecture



4.2 Request flow

1. PlatformDataProvider mounts and calls getPlatformSnapshot().
2. The snapshot probes all five services **in parallel** (Promise.all) through the gateway and records status (UP/DOWN) and latency per service.
3. Each request carries an X-Correlation-Id header (UUID) so a UI action can be traced in gateway and service logs.
4. The raw datasets are passed to buildIntelligence() (pure function) which returns health, risk, maturity, gaps, heatmap, analytics and recommendations.
5. All pages consume the same memoised intelligence object; data refreshes automatically every 60 seconds or on demand.
6. If a service is down, the platform shows a “Degraded” indicator instead of failing completely.

5. Backend Microservices

5.1 Service catalogue

Service	Port	Base path	Entity / Table	Package
Policy Service	8080	/policies	Policy / policies	com.saksham.policyservice
Regulation Service	8081	/regulations	Regulation / regulations	com.saksham.regulationservice
Compliance Service	8083	/compliances	Compliance / compliances	com.saksham.complianceservice
Report Service	8084	/reports	Report / reports	com.saksham.reportservice
User Service	8085	/users	User / users	com.saksham.userservice
API Gateway	8082	routes all of the above	–	com.saksham.apigateway

5.2 REST endpoints (identical pattern per service)

Method	Path	Purpose
GET	/resource}	List all
POST	/resource}	Create (validated)
GET	/resource}/{id}	Get by id (404 via ResourceNotFoundException)
PUT	/resource}/{id}	Update
DELETE	/resource}/{id}	Delete

Resources: policies, regulations, compliances, reports, users.

5.3 API Gateway routing

```
spring.cloud.gateway.server.webmvc.routes[0] Path=/policies/** ->
http://localhost:8080
spring.cloud.gateway.server.webmvc.routes[1] Path=/regulations/** ->
http://localhost:8081
spring.cloud.gateway.server.webmvc.routes[2] Path=/compliances/** ->
http://localhost:8083
spring.cloud.gateway.server.webmvc.routes[3] Path=/reports/** ->
http://localhost:8084
spring.cloud.gateway.server.webmvc.routes[4] Path=/users/** ->
http://localhost:8085
```

CORS (CorsConfig): allowed origin `http://localhost:3000`, all headers, all methods.

5.4 Cross-cutting backend features

- **Validation:** Jakarta Bean Validation on entities.
- **Error handling:** `GlobalExceptionHandler` + `ResourceNotFoundException` (policy and regulation services).
- **DTOs:** `PolicyResponseDTO` in `policy-service`.
- **OpenAPI:** `OpenApiConfig`; Swagger UI at `http://localhost:<port>/swagger-ui.html`.
- **Database:** `jdbc:postgresql://localhost:5432/legal_compliance_db`, schema managed by Hibernate (`ddl-auto=update`).

6. Data Model

6.1 Runtime entities (source of truth = JPA entities)

Entity	Fields	Validation
Policy	id (Integer, identity), title, description	title, description @NotBlank
Regulation	id, regulationName, country, description	all @NotBlank
Compliance	id, policyId, regulationId, status, remarks	status, remarks @NotBlank
Report	id, reportName, reportType, status, generatedDate (String)	name, type, status @NotBlank
User	id, username, email, role	username, role @NotBlank; email @Email

6.2 Relationships

```
Policy (1) -----< Compliance >----- (1) Regulation
                        |
                        status + remarks  -> scored by Risk Engine
Report  : evidence / reporting artefact (independent)
User    : role used for persona-based views
```

Schema drift: database/schema.sql describes an older design (compliance_checks with risk_score, created_at timestamps, users.password). The running services use the JPA entities above. Because the live data has no risk score and no timestamps, risk is *computed* by the frontend Risk Engine, and assessments in the timeline are ordered by record id. Introducing Flyway migrations is the recommended fix (see Roadmap).

7. Frontend Architecture

7.1 Folder structure

```
legal-compliance-ui/src
├─ api/
│   └─ httpClient.js           Shared Axios instance (base URL, correlation id,
error normalisation)
├─ config/
│   └─ navigation.js          Sidebar sections, routes, icons, allowed roles
├─ context/
│   └─ ThemeContext.jsx       Light/dark theme (localStorage + prefers-color-
scheme)
│   └─ RoleContext.jsx        Persona: Executive / Compliance Officer / Auditor
/ Admin
│   └─ PlatformDataContext.jsx Parallel data load, 60s auto-refresh, memoised
intelligence
├─ engine/
│   └─ riskEngine.js          Risk scoring, health, maturity, gaps, heatmap,
recommendations
│   └─ workflowEngine.js      Policy approval state machine
│   └─ copilotEngine.js       Grounded local answer engine
│   └─ engines.test.js        Unit tests
├─ services/
│   └─ dashboardService.js     getPlatformSnapshot() – service probes + data
│   └─ assistantService.js     Copilot gateway: local mode or remote RAG
│   └─ policy/regulation/compliance/report/userService.js
├─ layouts/
│   └─ AppShell.jsx            Sidebar + Topbar + Outlet + Palette + Copilot, %K
shortcut
│   └─ Sidebar.jsx             Role-filtered, collapsible navigation
│   └─ Topbar.jsx              Breadcrumb, search, live status, refresh, theme,
Copilot, role
├─ components/
│   └─ CommandPalette.jsx      Global search across all entities and pages
│   └─ CopilotDrawer.jsx       AI Copilot chat drawer with citations
│   └─ RequireRole.jsx         Route guard (UI-level)
│   └─ charts/Charts.jsx       Gauge, DonutChart, BarList, RadarChart (SVG)
│   └─ ui/Icon.jsx, ui/Primitives.jsx PageHeader, Panel, KpiCard, badges,
empty/loading states
├─ pages/
│   └─ CommandCenter.jsx Analytics.jsx RiskHeatmap.jsx GapAnalysis.jsx
│   └─ PolicyWorkflow.jsx AuditTimeline.jsx ExecutiveReport.jsx
│   └─ Policies.jsx Regulations.jsx Compliances.jsx Reports.jsx Users.jsx
(register)
├─ styles/theme.css            Design tokens, layout, components, dark mode,
print styles
└─ App.js                     Providers + route table generated from navigation
```

```
config
└─ setupTests.js                                jest-dom + TextEncoder polyfill
```

7.2 Provider hierarchy

```
ThemeProvider
└─ RoleProvider
    └─ PlatformDataProvider
        └─ BrowserRouter
            └─ AppShell (layout route)
                └─ RequireRole(roles) -> Page
```

7.3 Design principles

- **Separation of concerns:** engines are framework-free pure functions; React only renders.
- **Single source of truth:** one data snapshot, one intelligence object shared by every page.
- **Configuration-driven navigation:** routes, sidebar and role access come from `navigation.js`.
- **Graceful degradation:** per-service health probing; partial data still renders.
- **Design tokens:** CSS variables drive light/dark themes and chart colours.
- **Accessibility:** focus-visible outlines, ARIA labels, reduced-motion support, colour + text for status.

8. Feature Catalogue

#	Feature	Route / Location	What it does
1	Executive Command Center	/	Role-specific KPIs, Health gauge, maturity level, assessment status donut, top risks, AI recommendations, residual risk by regulation. Admin view shows live microservice health and users by role.
2	Compliance Health Score	Command Center, Report	0–100 score = 100 – average residual risk; bands Strong / Watch / Critical.
3	Risk Scoring Engine	engine/riskEngine.js	Scores every assessment from status, remark signals, jurisdiction and framework; returns human-readable drivers.
4	Risk Heatmap	/risk-heatmap	Policy × Regulation matrix coloured by risk level; empty hatched cells are coverage gaps; click a cell for drill-down with “why this score”.
5	Compliance Analytics	/analytics	Maturity radar + dimension bars, status donut, risk-level donut, risk by regulation, regulatory footprint, report pipeline. Jurisdiction cross-filter recomputes all charts.
6	Compliance Maturity Index	/analytics	Five-level CMMI-style index from four weighted dimensions with next-level guidance.
7	Gap Analysis	/gap-analysis	Uncovered regulations, unassessed policies, failing controls, data-integrity (orphan) issues.
8	AI Recommendations	Command Center, Copilot, Report	Prioritised P1–P3 actions ranked by expected health uplift.
9	Policy Approval Workflow	/workflow	Kanban across Draft → Legal Review → Compliance Review → Executive Approval → Published (+ Rejected). Role-gated transitions, risk control gate, approval trail.
10	Audit Timeline	/timeline	Unified evidence trail of approvals, reports and assessments with filters.
11	Executive PDF Report	/executive-report	Board briefing: narrative summary, KPIs, material risks, recommendations, methodology, sign-off block. Export via browser print to PDF.
12	AI Compliance Copilot	Top bar “Copilot”	Chat drawer with starter prompts, grounded answers, clickable items, numbered citations, follow-ups.
13	Global Search / Command Palette	⌘K / Ctrl+K	Search pages, policies, regulations, risks, reports, users (admin); keyboard navigation; “Ask Copilot”

			fallback.
14	Role-Based Dashboards	Top bar role selector	Executive, Compliance Officer, Auditor, Administrator personas with different KPIs, navigation and access.
15	Dark Mode	Top bar moon/sun	Bootstrap 5.3 theme switching persisted to localStorage.
16	Enterprise Shell	layouts/	Collapsible sidebar, sticky blurred top bar, breadcrumb, responsive mobile drawer.
17	Live Platform Status	Top bar pill, Admin view	Live/Degraded indicator, last sync time, per-service latency.
18	Registers	/policies, /regulations, /compliances, /reports, /users	Existing data modules rendered inside the new shell.

9. Intelligence Engines (Algorithms & Formulas)

9.1 Status classification

Status text is normalised to lower case and matched in order (non-compliant is tested before compliant):

Pattern (regex)	Class	Severity
(non not)[_]?compliant fail violat breach	NON_COMPLIANT	1.0
partial	PARTIAL	0.6
pending progress review open draft	UNDER_REVIEW	0.4
compliant pass approved closed complete	COMPLIANT	0.0
(anything else)	UNKNOWN	0.5

9.2 Remark risk signals

Pattern	Signal	Weight
breach leak incident	Incident indicator	+12
penalt fine sanction enforcement	Regulatory penalty exposure	+10
critical urgent severe	Escalated severity	+8
expired overdue outdated lapsed	Lapsed control	+6
missing absent no evidence not documented gap	Missing evidence	+6

9.3 Multipliers

Factor	Rule	Multiplier
Jurisdiction	EU / European Union	×1.20
Jurisdiction	USA / US / United States / UK / United Kingdom	×1.10
Jurisdiction	India	×1.05
Framework	GDPR, HIPAA, SOX, PCI, DPDP, CCPA, Basel, DORA, NIS2, ISO 27001 in regulation name	×1.15

9.4 Residual risk score

```
if status = COMPLIANT:
    risk = 0
else:
```

```
raw = (severity × 70 + ∑ signal weights) × jurisdiction × framework
risk = clamp(round(raw), 1, 100)
```

Example: NON_COMPLIANT GDPR (EU) with remarks “Breach risk, penalty exposure” → $(70 + 12 + 10) \times 1.2 \times 1.15 = 126.96 \rightarrow \mathbf{100 \text{ (Critical)}}$.

Every score returns a **drivers** list (e.g. “Status: Non-Compliant”, “Incident indicator in remarks (+12)”, “Jurisdiction weight EU (x1.2)”) – this is shown in the heatmap drill-down as “Why this score”.

9.5 Risk levels

Level	Score	Colour
Critical	≥ 85	Dark red
High	65 – 84	Red
Medium	40 – 64	Amber
Low	< 40	Green

9.6 Compliance Health Score

```
Health = round(100 - mean(risk of all assessments))      (null when there are no assessments)
Bands:  ≥ 85 Strong   |   70–84 Watch   |   < 70 Critical
```

9.7 Coverage and rates

```
Policy coverage      = policies with ≥1 assessment / total policies
Regulation coverage  = regulations with ≥1 assessment / total regulations
Compliance rate      = COMPLIANT assessments / total assessments
```

9.8 Compliance Maturity Index

Dimension	Definition	Weight
Control Coverage	average(policy coverage, regulation coverage)	30%
Control Effectiveness	compliance rate	40%
Assessment Completeness	$1 - (\text{under review} / \text{assessments})$	15%
Reporting Cadence	$\min(1, \text{reports} / \text{regulations})$	15%

```
Index = round(100 × ∑ weight × value)
```

Level	Name	Index $\geq$	Meaning
-------	------	--------------	---------

1	Initial	0	Ad-hoc controls with limited visibility
2	Repeatable	20	Core controls exist but coverage is inconsistent
3	Defined	40	Standardised assessments across key regulations
4	Managed	60	Quantitatively measured and reported compliance
5	Optimizing	80	Continuous, data-driven improvement

The weakest dimension is reported as the focus area for reaching the next level.

9.9 Gap Analysis

- **Uncovered regulations** – no assessment; exposure High if a high-impact framework, otherwise Medium.
- **Unassessed policies** – policy never tested against any regulation.
- **Failing controls** – assessments with risk ≥ 65 (High/Critical).
- **Data-integrity issues** – assessments referencing a policy or regulation id that does not exist.

9.10 Recommendation engine

Source	Priority	Uplift
Each open finding (risk > 0)	P1 if ≥ 65 , P2 if ≥ 40 , else P3	round(risk / number of assessments) health points
Uncovered regulation	P1 (high-impact framework) / P2	–
Unassessed policy	P3	–
Next maturity level	P2	–

Sorted by priority, then by uplift (largest first).

9.11 Policy Approval Workflow (state machine)

From	To	Action	Allowed roles	Gate
DRAFT	LEGAL_REVIEW	Submit for Legal Review	Compliance Officer, Admin	–
LEGAL_REVIEW	COMPLIANCE_REVIEW	Legal Sign-off	Compliance Officer, Admin	–

COMPLIANCE_REVIEW	EXECUTIVE_APPROVAL	Compliance Sign-off	Compliance Officer, Admin	Blocked while any linked assessment has risk $\geq$ 65
EXECUTIVE_APPROVAL	PUBLISHED	Approve & Publish	Executive, Admin	–
LEGAL / COMPLIANCE REVIEW	REJECTED	Reject	Compliance Officer, Admin	–
EXECUTIVE_APPROVAL	REJECTED	Reject	Executive, Admin	–
REJECTED	DRAFT	Reopen as Draft	Compliance Officer, Admin	–

Initial stage is derived from evidence: no assessments → DRAFT; all compliant → PUBLISHED; otherwise → COMPLIANCE_REVIEW. Manual decisions (with actor, role and timestamp) are stored in localStorage key lca.workflow.v1 and appear in the approval trail and Audit Timeline.

9.12 Copilot engine (local intelligence mode)

Order of resolution: (1) entity match – a policy title or regulation name in the question; (2) intent match; (3) help response.

Intent	Trigger words	Answer
Summary	summary, brief, executive, board, overview, posture	Executive brief: health, maturity, risks, coverage, top 3 recommendations
Health	health, score, how are we, compliant are	Health score, method, compliant vs open findings
Maturity	matur, cmmi, level	Level, weakest dimension, all dimensions
Gaps	gap, uncovered, missing, not covered, unassessed	Uncovered regulations, unassessed policies, failing controls
Recommend	recommend, what should, next step, fix, remediates, priorit, action	Top 5 recommendations with uplift
Risk	risk, critical, non-compliant, fail, violation, breach, exposure	High/Critical findings with citations

Every answer contains: title, summary lines, clickable items (navigate to the relevant page), numbered citations, follow-up suggestions, and a grounded: true flag.

9.13 RAG-ready contract (future)

REACT_APP_ASSISTANT_MODE=remote

POST /assistant/ask

Request : { "question": "...", "context": { "healthScore": 72, "maturityLevel": 3 } }

Response: { "title": "...",
 "summary": ["..."],
 "items": [{ "label": "...", "detail": "...", "tone": "danger",
"route": "/risk-heatmap" }],
 "citations": [{ "label": "GDPR Art.32", "route": "/regulations" }] }

On failure the UI falls back to the local engine and shows “RAG service unavailable”.

10. Role-Based Views and Access Matrix

Page	Executive	Compliance Officer	Auditor	Admin
Command Center	✓	✓	✓	✓
Compliance Analytics	✓	✓	✓	✓
Risk Heatmap	✓	✓	✓	✓
Gap Analysis	–	✓	✓	✓
Policy Approvals	✓	✓	–	✓
Audit Timeline	–	✓	✓	✓
Executive Report	✓	–	✓	✓
Policies / Regulations / Assessments / Reports	✓	✓	✓	✓
Users & Access	–	–	–	✓

Command Center KPIs per role

Role	Title	KPIs
Executive	Executive Command Center	Health Score, Maturity Index, High-Risk Findings, Regulatory Coverage
Compliance Officer	Compliance Operations	Open Findings, Under Review, Unassessed Policies, Uncovered Regulations
Auditor	Assurance Workspace	Assessments, Total Gaps, Integrity Issues, Reports on File
Administrator	Platform Operations	Services Online, Users, Records Managed, Health Score + Microservice Health panel

The role selector is a **UI persona**. Real authorization must be enforced at the API Gateway with JWT role claims (Spring Security / Keycloak).

11. Configuration Reference

11.1 Environment variables (frontend, .env)

Variable	Default	Purpose
REACT_APP_API_BASE_URL	http://localhost:8082	API Gateway base URL
REACT_APP_ASSISTANT_MODE	local	remote sends Copilot questions to /assistant/ask

11.2 Browser storage keys

Key	Content
lca.theme	light dark
lca.role	EXECUTIVE COMPLIANCE_OFFICER AUDITOR ADMIN
lca.workflow.v1	Policy workflow stages and approval history (JSON)

11.3 Platform constants

- Auto-refresh interval: 60 seconds (PlatformDataContext.jsx).
- HTTP timeout: 10 seconds (httpClient.js).
- Command palette: max 12 results.

12. Running the POC Locally

12.1 Prerequisites

- Java 21, Maven wrapper (included), Node.js 18+ and npm, PostgreSQL running on 5432.
- Database legal_compliance_db created; datasource username set in each service's application.properties.

12.2 Start order

```
# 1. Database
pg_isready

# 2. Microservices (one terminal each)
cd services/policy-service      && bash mvnw spring-boot:run    # :8080
cd services/regulation-service  && bash mvnw spring-boot:run    # :8081
cd services/compliance-service  && bash mvnw spring-boot:run    # :8083
cd services/report-service      && bash mvnw spring-boot:run    # :8084
cd services/user-service        && bash mvnw spring-boot:run    # :8085

# 3. Gateway
cd services/api-gateway         && bash mvnw spring-boot:run    # :8082

# 4. Frontend
cd legal-compliance-ui && npm install && npm start                # :3000
```

12.3 Useful URLs

- Application: <http://localhost:3000>
- Gateway sample: <http://localhost:8082/policies>
- Swagger UI: <http://localhost:8080/swagger-ui.html> (replace port per service)

12.4 Sample data that demonstrates the engine well

```
Regulations: GDPR (EU), HIPAA (USA), IT Act (India), SOX (USA)
Compliances:
  policyId=1, regulationId=GDPR,  status=NON_COMPLIANT, remarks="Breach risk,
penalty exposure"
  policyId=2, regulationId=GDPR,  status=PARTIAL,      remarks="Missing MFA
evidence"
  policyId=2, regulationId=IT Act,status=COMPLIANT,    remarks="Evidence verified"
  policyId=3, regulationId=SOX,   status=PENDING,     remarks="Awaiting review"
Leave one regulation and one policy unmapped to show coverage gaps.
```

13. Testing and Quality

```
cd legal-compliance-ui
CI=true npx react-scripts test --watchAll=false
npx react-scripts build
```

Suite	Tests	Covers
engine/engines.test.js	9	Status classification order; jurisdiction/framework/signal weighting and drivers; zero risk for compliant; health, coverage, gaps, recommendations; empty platform; workflow derivation, role gating and control gate; approval trail; copilot grounding and entity answers
App.test.js	1	Smoke test: app shell and Executive Command Center render (Axios mocked)

Result at time of writing: 2 suites, 10 tests passing; production build compiled with no ESLint warnings.

Test configuration notes: `package.json` maps `react-router/dom` for Jest (React Router v7 uses `package exports`), and `setupTests.js` polyfills `TextEncoder/TextDecoder` for `jsdom`.

14. Known Limitations and Honest Disclosures

Area	Limitation	Recommended fix
Risk scoring	Computed in the browser; not persisted	Move to an intelligence-service; store risk snapshots
Copilot	Rule-based, grounded local engine – not an LLM	Spring AI + pgvector RAG service implementing /assistant/ask
Workflow	Decisions stored in browser localStorage	policy_approvals table + transition endpoint enforced server-side
Security	No authentication; role switcher is UI only	Keycloak/OIDC + JWT validation at the gateway
Timeline	Assessments lack timestamps; ordered by id	Audit columns (createdAt/updatedAt) + audit_log table
Schema	schema.sql differs from JPA entities; ddl-auto=update	Flyway versioned migrations
Forecasting	No historical data	risk_snapshots table captured nightly
Unused files	pages/Dashboard.jsx and components/Navbar.jsx no longer routed	Delete when convenient

15. Demo Script (5–7 minutes)

1. **Executive persona** – open Command Center: health gauge, maturity level, KPIs, top risks.
2. Click “**Brief me**” – Copilot produces a grounded executive summary with citations.
3. Open **Risk Heatmap** – click a red cell – show “Why this score” drivers; click a hatched cell – coverage gap.
4. Open **Analytics** – change jurisdiction filter – all charts recompute; show maturity radar.
5. Switch to **Compliance Officer** – Policy Approvals – try Compliance Sign-off on a risky policy – **control gate blocks it**.
6. Switch to **Auditor** – Gap Analysis (integrity issues) and Audit Timeline with approval trail.
7. Switch to **Admin** – live microservice health with latency; Users & Access.
8. Press ⌘K – search “GDPR” – choose “Ask Copilot”.
9. Toggle **dark mode**; resize to mobile.
10. Open **Executive Report** – Export PDF.

16. Interview Talking Points and Q&A

16.1 Talking points

- **Microservices + API Gateway:** single entry point, CORS centralised, independent deployability per domain.
- **Explainable risk:** deterministic, unit-tested scoring with drivers – auditors can reproduce any number.
- **Resilience:** parallel probing with per-service status; degraded mode instead of blank screens.
- **Observability-ready:** correlation id on every request.
- **Separation of concerns:** pure engines vs. React rendering; easy to migrate engines server-side.
- **Governance by design:** maker-checker workflow, role gating, risk control gate, approval trail.
- **AI readiness:** grounded answers with citations today; documented RAG contract for tomorrow.
- **Zero added dependencies:** custom SVG charts and icons keep the bundle lean and theme-aware.

16.2 Likely questions

Question	Answer outline
Why compute risk on the client?	The current data model has no risk field; computing client-side proved the model quickly. Engines are pure functions, so moving them to an intelligence-service is a lift-and-shift.
How do you prevent the Copilot from hallucinating?	Local mode only uses platform data and always cites records. In RAG mode: permission-filtered retrieval, mandatory citations, refusal on low evidence, evaluation (faithfulness) in CI.
How is RBAC enforced?	Today UI-only. Production: Keycloak OIDC, JWT validated at the gateway, role claims checked per route and in services; row-level security in PostgreSQL for tenancy.
What happens if a service is down?	Promise.all over individually-caught probes; the service is marked DOWN, its data is empty, the UI shows “Degraded”, everything else works.
How would you scale this?	Kubernetes + HPA, outbox/Kafka events, caching at the gateway, read models for analytics, OpenTelemetry tracing.
Why a maturity index?	Executives need a trajectory, not just a snapshot; CMMI-style levels are familiar to boards and auditors.

17. Future Roadmap – Next-Generation GRC + AI Platform

17.1 Ranked capabilities (by ROI and impact)

Rank	Capability	Focus areas	Effort
1	Compliance Data Fabric & Obligation Model	Data fabric	M
2	RAG Knowledge Copilot (LLM + Vector Search)	RAG, vector search, LLM	M
3	Document Intelligence	Document intelligence	M
4	Regulatory Change Impact Analysis	Regulatory change	M
5	Real-Time Event Backbone & Autonomous Monitoring	Monitoring, real-time events	M–L
6	Multi-Agent Agentic Workflows (human-in-the-loop)	AI agents, multi-agent, agentic workflows	L
7	Compliance Knowledge Graph	Knowledge graph, intelligence	M
8	Policy Recommendation & Drafting Engine	Policy recommendation	M
9	Audit Intelligence & tamper-evident evidence	Audit intelligence	M
10	Predictive Compliance & Risk Forecasting	Forecasting, predictive	M
11	Compliance Digital Twin & Executive Decision Support	Digital twin, decision support	M
–	Cloud-Native Enterprise Platform (continuous)	Cloud-native	L

17.2 Capability details

1. Compliance Data Fabric & Obligation Model

- **Value:** Regulation → Obligation → Control → Policy → Evidence → Assessment traceability; prerequisite for everything else.
- **Backend:** audit columns and @Version on all entities; Flyway; transactional outbox; risk engine moved to intelligence-service.
- **Database:** obligations, controls, control_obligation_map, policy_control_map, evidence, risk_snapshots, outbox_events.
- **Stack:** Flyway, Debezium / Spring Modulith events, JPA auditing, PostgreSQL RLS.

2. RAG Knowledge Copilot

- **Value:** cited answers across regulations, policies and evidence in seconds.
- **Architecture:** ai-orchestrator-service implementing /assistant/ask; hybrid retrieval (vector + full-text, reciprocal rank fusion) with ACL filtering.

- **Backend:** Spring AI ChatClient + QuestionAnswerAdvisor; Kafka-driven indexer (≈800-token chunks, 100 overlap); SSE streaming; PII redaction and prompt-injection guard.
- **Database:** doc_chunks(..., embedding vector(1536), tsv tsvector, acl jsonb) with HNSW + GIN; ai_interactions.
- **Stack:** Spring AI, pgvector, Azure OpenAI or Ollama (llama3.1, nomic-embed-text), RAGAS evaluation in CI.

3. Document Intelligence

- **Value:** upload a regulation/policy PDF → extracted obligations, deadlines, penalties → suggested control mappings.
- **Backend:** document-service; Tika/PDFBox/Tesseract or Azure Document Intelligence; Spring AI structured output (ObligationCandidate); confidence < 0.75 → review queue.
- **Frontend:** upload wizard; split PDF/extraction review view.
- **Database:** documents, obligation_candidates.

4. Regulatory Change Impact Analysis

- **Value:** “Regulation amended → 7 policies, 12 controls, 3 owners affected → draft remediation plan.”
- **Backend:** semantic clause diff (ADDED/REMOVED/MODIFIED); impact score = materiality × affected controls × current risk; RegulatoryChangeDetected event.
- **Frontend:** Change Radar with clause diff and impact graph.
- **Database:** regulation_versions, obligation_changes, impact_assessments.

5. Real-Time Event Backbone & Autonomous Monitoring

- **Value:** continuous control monitoring instead of point-in-time audits.
- **Backend:** Kafka topics (policy, compliance, regulation, control.signals, alerts); monitoring-service with declarative rules (e.g. evidence > 90 days, risk +20 in 24h); SSE push.
- **Database:** monitoring_rules, alerts (with fingerprint de-duplication).
- **Stack:** Kafka/Redpanda, Spring Cloud Stream, Debezium, Drools, Resilience4j.

6. Multi-Agent Agentic Workflows

- **Agents:** Supervisor → Regulatory Analyst, Control Assessor, Policy Drafter, Audit Agent, Reporter.
- **Governance:** tools = existing REST APIs (Spring AI @Tool / MCP); read tools autonomous, write tools require human approval; agent acts with the user’s identity; token budgets.
- **Frontend:** Agent Console with live step trace and approval cards.
- **Database:** agent_runs, agent_steps.

7. Compliance Knowledge Graph

- **Model:** nodes Regulation, Obligation, Control, Policy, Evidence, Owner, Risk, System; edges REQUIRES, SATISFIED_BY, IMPLEMENTS, PROVES, OWNS, AFFECTS.
- **Uses:** impact analysis, “test once comply many”, GraphRAG context expansion.

- **Stack:** Apache AGE (inside PostgreSQL) or Neo4j; React Flow / Cytoscape.js.

8. Policy Recommendation & Drafting Engine

- Gap → AI-drafted, cited clause → redline vs. current policy → existing approval workflow.
- Database: policy_versions (author_type HUMAN/AI), clause_library (with embeddings).

9. Audit Intelligence

- Hash-chained, append-only audit_log; segregation-of-duties anomaly detection; statistical sampling; one-click audit pack (PDF + ZIP with SHA-256 manifest).

10. Predictive Compliance & Risk Forecasting

- Requires 8–12 weeks of risk_snapshots. Phase A: Holt-Winters forecasting; Phase B: gradient-boosted “control fails in 90 days” classifier with SHAP explanations.
- Database: predictions, model_registry. Stack: Tribuo/Smile (Java) or FastAPI + LightGBM + MLflow.

11. Compliance Digital Twin & Executive Decision Support

- Snapshot the model, apply scenario mutations (remediate controls, add regulation), re-run engines, compare before/after; optimiser suggests best remediation for a budget.
- Database: scenarios, decisions. Stack: pure Java domain engine, Timefold (OptaPlanner).

Cloud-Native Enterprise Platform (cross-cutting)

- Keycloak OIDC; JWT at gateway; OAuth2 client-credentials between services.
- Multi-tenancy via tenant_id + PostgreSQL row-level security.
- OpenTelemetry → Grafana / Tempo / Prometheus / Loki; LLM token, cost and quality metrics.
- Docker Compose (dev), Kubernetes + Helm + Argo CD (prod), Vault for secrets, HPA on AI services.
- CI: GitHub Actions, Testcontainers, RAG evaluation gate, OWASP dependency check, signed images + SBOM.
- AI governance: prompt registry, model allow-list, PII redaction, EU AI Act use-case register.

17.3 Target architecture

```

React Platform UI --(JWT, SSE)--> API Gateway (Spring Cloud Gateway + Keycloak)
    |-- Policy / Regulation / Compliance / Report / User services --(Outbox)-->
Kafka
    |-- Intelligence Service (risk, maturity, forecasting, digital twin)
    |-- AI Orchestrator (Spring AI agents + RAG) --> Azure OpenAI / Ollama
    |-- Document Service (Tika / Azure Document Intelligence) --> Kafka
Kafka --> Monitoring Service (rules, alerts --SSE--> UI)
Kafka --> Indexer (chunk + embed) --> PostgreSQL + pgvector | Apache AGE graph
Observability: OpenTelemetry --> Grafana / Tempo / Loki / Prometheus

```

17.4 Delivery phases

Phase	Scope
Foundation	Data fabric, Flyway, outbox, Keycloak
AI Core	RAG Copilot + pgvector; Document Intelligence
Differentiators	Knowledge graph; Regulatory change impact; Event backbone + monitoring
Agentic	Multi-agent + human approval; Policy drafting engine
Foresight	Audit intelligence; Forecasting + Digital twin

Highest-ROI demo path: 1 → 2 → 3 → 4 — upload a regulation PDF, obligations are extracted, the Copilot answers with clause citations, Change Radar shows impacted policies via the graph, an agent drafts remediation, a human approves it in the existing workflow, and the health score updates live.

18. File Inventory

18.1 Frontend (legal-compliance-ui/src)

File	Responsibility
App.js	Providers and role-guarded route table generated from navigation config
api/httpClient.js	Axios instance: base URL, 10s timeout, X-Correlation-Id, error normalisation
config/navigation.js	Sidebar sections, routes, icons, roles
context/ThemeContext.jsx	Dark/light theme
context/RoleContext.jsx	Persona roles and hasRole()
context/PlatformDataContext.jsx	Snapshot loading, auto-refresh, intelligence memoisation, degraded flag
engine/riskEngine.js	classifyStatus, scoreAssessment, riskLevel, healthBand, buildIntelligence, recommendations, maturity
engine/workflowEngine.js	STAGES, TRANSITIONS, deriveStage, availableTransitions, applyTransition
engine/copilotEngine.js	answerLocally – intents, entity answers, citations
engine/engines.test.js	Unit tests for all engines
services/dashboardService.js	PLATFORM_SERVICES, getPlatformSnapshot, getDashboardData
services/assistantService.js	askCopilot – local/remote mode with fallback
services/*Service.js	Per-register API calls via shared client
layouts/AppShell.jsx, Sidebar.jsx, Topbar.jsx	Enterprise shell
components/CommandPalette.jsx	Global search (⌘K)
components/CopilotDrawer.jsx	Copilot chat UI
components/RequireRole.jsx	Route guard
components/charts/Charts.jsx	Gauge, DonutChart, BarList, RadarChart
components/ui/Icon.jsx, Primitives.jsx	Icons and UI building blocks
pages/*.jsx	CommandCenter, Analytics, RiskHeatmap, GapAnalysis, PolicyWorkflow, AuditTimeline, ExecutiveReport + registers
styles/theme.css	Design tokens, components, dark mode, responsive and print styles

18.2 Backend (services/)

Path	Content
api-gateway/	ApiGatewayApplication, CorsConfig, route configuration
policy-service/	Policy entity, PolicyResponseDTO, PolicyRepository, PolicyService, PolicyController, GlobalExceptionHandler, OpenApiConfig
regulation-service/	Regulation entity, controller, service, repository, exception handling, OpenAPI
compliance-service/	Compliance entity, controller, service, repository
report-service/	Report entity, controller, service, repository
user-service/	User entity, controller, service, repository, OpenAPI
database/schema.sql	Original (older) schema design

19. Glossary

Term	Meaning
GRC	Governance, Risk and Compliance
Assessment	A Compliance record linking a policy to a regulation with a status and remarks
Residual risk	Risk remaining after current controls, 0–100
Health Score	100 – average residual risk
Maturity Index	Weighted 0–100 score mapped to CMMI-style levels 1–5
Coverage gap	Regulation or policy with no assessment
Control gate	Workflow rule blocking approval while high risks are open
Maker-checker	Separation between the person who prepares and the person who approves
RAG	Retrieval-Augmented Generation – LLM answers grounded in retrieved documents
Vector search	Similarity search over embeddings (pgvector)
Outbox pattern	Writing domain events in the same DB transaction, then publishing them reliably
Correlation ID	Unique request id propagated across services for tracing
CMMI	Capability Maturity Model Integration
HITL	Human-in-the-loop approval of AI actions